Constrained Edge Pruning and Trust-Aware GNN Inference for Task Allocation in Dynamic Rover Networks: a QUBO Formulation with a Solver-Level Audit

Anonymous Authors
`quantum-gnn` project

October 20, 2026

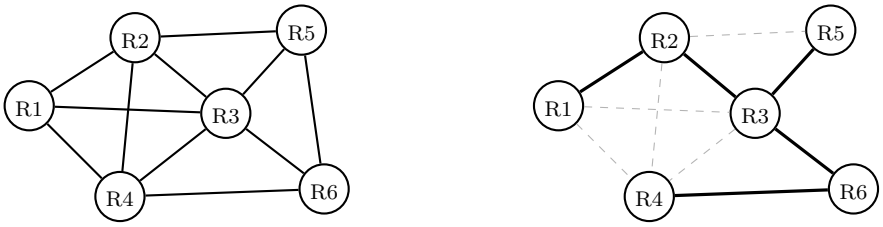
Abstract

Rovers that coordinate over low-power radio see a communication graph that changes from one cycle to the next. We study a pipeline that allocates tasks over such a graph in two steps. A constrained quadratic unconstrained binary optimization (QUBO) model first prunes the graph to a working subgraph that respects per-rover link caps, a processing budget and exact connectivity of the mission rovers. A lightweight trust-aware graph neural network (GNN) then scores the rovers on that subgraph, and trust is learned online from task outcomes. Our main interest is in checking how well each part works. The encoding agrees with an independent re-derivation to within 2.3×10^{-10}. On the actual Q matrices, four annealing-family samplers (local simulated annealing, D-Wave's `neal`, tabu search and simulated quantum annealing) find the exact optimum on only 48 to 64% of verifiable instances, because the penalties that make violations unprofitable also block the samplers' moves. A feasibility-projected annealer is exact on all of them and matches exact mixed-integer optima on 95% of larger instances, but the mixed-integer program solved directly by CBC is also exact and about 100 times faster. On classical hardware the QUBO is therefore best seen as an encoding for annealing hardware. In a closed-loop simulator where mission success depends causally on the kept links, trust-aware assignment beats a trust-only baseline by 8.1 percentage points (paired bootstrap 95% CI $[+4.2, +11.7]$, $p = 0.016$) and random assignment by 13.8 points. The QUBO pruner is the only arm that never violates a constraint, but it gives up 2 to 4 points of raw success against constraint-aware repaired baselines, and no level of a ten-level constraint-pressure sweep reverses this. Logging which constraints were enforced also showed that our own earlier comparison used a budget that could never be met. Finally, the unchanged pipeline runs in a Gazebo and ROS 2 world and drives an interactive 3D simulation engine.

1 Introduction

Small, inexpensive rovers are attractive for survey, inspection and monitoring work in places where there is no fixed communication infrastructure. A team of such rovers has to organize itself. Every few seconds some node must decide which rover takes which task, and that decision can only be as good as the information that reaches it. In our setting the information travels over ESP-NOW links between ESP32-class boards. Each rover hears only a few teammates, and every link changes as rovers move, batteries drain and terrain gets in the way. The communication network is therefore a graph whose edges appear, weaken and disappear from cycle to cycle.

Two problems follow. Running GNN inference on the full graph wastes the small compute budget of the fog device and lets poor links distort the scores. At the same time, simple pruning rules that judge one edge at a time cannot respect constraints that depend on the whole kept graph, such as degree limits, a processing budget, or the requirement that mission-critical rovers stay connected. We address both by pruning with a constrained QUBO, the input format of



(a) candidate graph, 11 links (b) kept working graph, 5 links

Figure 1: The pruning step. Out of the links that are in range (a), a small connected subset is kept (b, solid lines) under link caps, a cost budget and connectivity of the mission rovers.

quantum and quantum-inspired annealers, and then assigning tasks with a trust-aware GNN on the kept graph (Fig. 1).

We started with the aim of building a quantum-ready pipeline. Along the way the more useful outcome turned out to be a careful account of what each component contributes when it is measured against the strongest alternatives we could find, including ones that beat our pipeline. The contributions are:

- A constrained QUBO for dynamic graph pruning with utility, stability, degree, budget and connectivity terms, in which connectivity is enforced exactly by a single-commodity flow encoding. The implementation is checked against an independent re-derivation and exhaustive enumeration, and a five-rover example shows the connectivity trap it avoids (Section 4).
- A solver audit on the actual Q matrices. Four annealing-family samplers miss a third or more of verifiable optima because of a penalty barrier that we quantify. A feasibility-projected annealer removes the barrier, and an exact MILP baseline solves the problem directly and faster than any annealer (Section 5).
- A trust-aware assignment layer with an online trust model, aging, resource gates and delivery awareness, evaluated against trust-only, no-graph and random baselines with paired statistics (Sections 6 and 7.3).
- A causal evaluation that includes constraint-aware repaired baselines, delivery-critical precision and recall, a constraint-pressure sweep, cloud amortization, scaling and fault recovery, together with an audit note on a budget that was silently relaxed in our earlier results (Section 7).
- Two deployments of the unchanged pipeline: a containerized Gazebo Harmonic and ROS 2 Jazzy world, and a browser-based 3D simulation engine with collision-free differential-drive motion and live paired comparisons (Section 8).

2 Related Work

Graph neural networks and graph pruning. Kipf and Welling [1] made graph convolutional networks popular and Hamilton et al. [2] extended them to inductive settings. Several works prune the input graph or the weights of a GNN. STEP [3] learns a self-supervised pruner for large dynamic graphs from graph statistics alone. CGP [4] sparsifies structure, features and weights together during training, Wang et al. [5] remove edges according to their effect on the loss, and Peng et al. [6] study sparse training of GNN weights. These methods aim at model or training efficiency on static benchmarks and do not enforce hard constraints. Spectral sparsification [7] preserves the spectrum of a graph but also has no notion of degree caps, budgets or a required set.

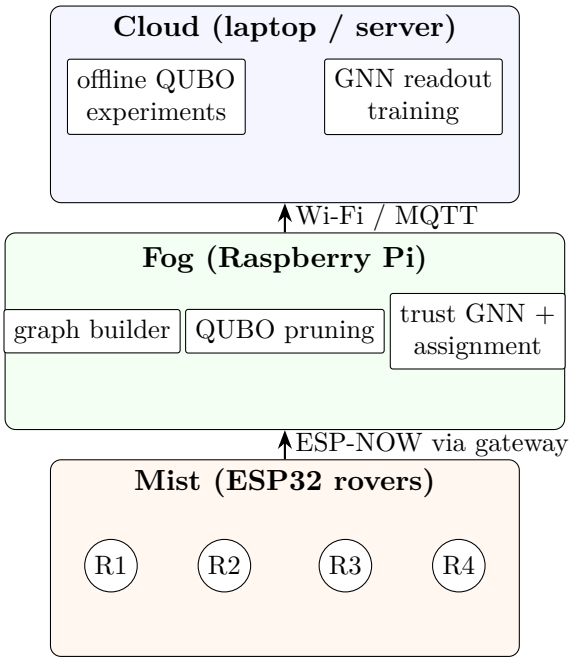


Figure 2: Three-tier deployment. Telemetry goes up and task assignments come back down. Solving and training run in the cloud or on the fog device, depending on how much time a cycle allows.

QUBO models and their solvers. Glover et al. [8] and Lucas [9] survey QUBO and Ising formulations. Simulated annealing [10] is available in production form through D-Wave's Ocean tools [11], quantum annealing [12] is emulated classically by simulated quantum annealing in OpenJij [13], and tabu search [14] is a strong local-search alternative. Our exact baselines use the CBC branch-and-cut solver [15] and the integrality of single-commodity flow relaxations [16].

Task allocation, trust and fog computing. In the taxonomy of Korsah et al. [17] our problem has single-task robots, single-robot tasks and instantaneous assignment that is recomputed every cycle. Ahanger et al. [18] use a quantum-inspired optimizer to allocate IoT tasks across edge and fog nodes, and MAG-Q [19] combines graph attention, actor-critic learning and a QUBO forwarding module for offloading in clustered fog networks. Trust appears in load balancing for edge computing [20] and in UAV task offloading [21], and computational trust models are reviewed by Sabater and Sierra [22]. When trust is estimated without the graph, choosing a rover becomes a bandit problem [23], so we include a trust-only baseline. The mist, fog and cloud split follows Bonomi et al. [24].

As far as we know, constrained QUBO pruning has not been combined with online trust-aware GNN inference for rover task allocation before, and we are not aware of a solver-level comparison of penalty-encoded sampling, feasibility-projected annealing and direct MILP for this kind of problem.

3 System Model

The deployment has three tiers (Fig. 2). Rovers in the mist tier sense their state and exchange telemetry over ESP-NOW. A gateway forwards the telemetry to a Raspberry Pi in the fog tier, which maintains the graph, prunes it and assigns tasks within the cycle deadline. A laptop or server in the cloud tier retrains the readout and can take over expensive solves.

At cycle t the fleet is an undirected graph $G_t = (V, E_t)$ whose edges are the pairs of rovers that can currently exchange packets. Each edge $e = (i, j)$ carries normalized features: trust $T_e = \min(T_i^{t-1}, T_j^{t-1})$ from the previous cycle, link quality L_e, battery $B_e = \min(B_i, B_j)$, proximity

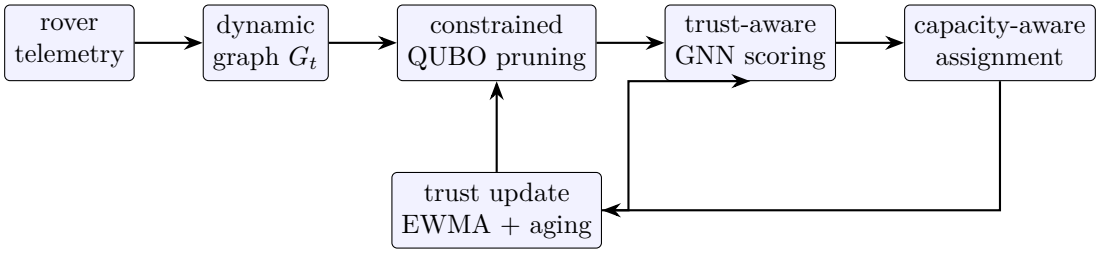


Figure 3: One pipeline cycle. Trust from the previous cycle enters both the pruner's edge features and the GNN's node features, so no stage depends on its own output within a cycle.

P_e, freshness $A_e = \exp(-\Delta t_e/3)$ since the link was last heard, and the criticality C_e of the tasks at its ends. A processing cost models transmission and queueing time, $c_e = \max(1, \lfloor 10\,\bar{c}_e \rceil)$ with $\bar{c}_e = 0.1 + 0.5(1 - P_e) + 0.15(\ell_i + \ell_j)$. The usefulness of an edge is the weighted utility

$$U_e = 0.25\,T_e + 0.25\,L_e + 0.10\,B_e + 0.10\,P_e + 0.15\,A_e + 0.15\,C_e, \tag{1}$$

which every pruning baseline also uses. Each rover has trust T_i, battery B_i, load ℓ_i and capability q_i.

Causal mission model. Pruning can only be judged if the kept links affect the outcome, so a task succeeds only when two events both happen. The assigned rover i *executes* the task with probability equal to its hidden reliability, provided its battery is above 15%. The result must then be *delivered* to the root rover r over the kept subgraph, which succeeds with probability

$$p_i = \max_{\pi:\, i \to r} \; \min_{e \in \pi} L_e, \tag{2}$$

the weakest link on the widest path from i to r ($p_r = 1$, and $p_i = 0$ when no path exists). Trust is updated from execution outcomes, so it estimates reliability, while delivery ties success to the pruning decision. An earlier version of our simulator had no delivery step, so success did not depend on the kept graph at all. Every end-to-end result in this paper uses the coupled model.

Each cycle runs the pipeline of Fig. 3: build the graph, prune it under the constraints, score the rovers, assign tasks greedily under per-rover capacity, observe outcomes and update trust. The mission-critical set is chosen once per scenario and anchors the connectivity constraint.

4 Constrained QUBO Formulation

Let H be the hard-critical edges, which are always kept, and $O = E_t \setminus H$ the optional edges. Let R be the required set with root r and $m = |R| - 1$, let D_i be the degree caps and K the budget. With a binary x_e for each optional edge, binary-encoded integer slacks s_i, s_C and q_e, and integer flows f_{uv} on directed arcs, the energy is

$$\begin{aligned} H(z) = &\sum_{e \in O} (-U_e + \lambda)\, x_e + \eta \sum_{e \in O} (x_e - x_e^{t-1})^2 \\ &+ \rho_d \sum_{i \in V} \Big(\sum_{e \in \delta_O(i)} x_e + s_i - (D_i - \deg_H(i)) \Big)^2 + \rho_C \Big(\sum_{e \in O} c_e x_e + s_C - (K - c_H) \Big)^2 \\ &+ \rho_F \sum_{v \in V} \big(\mathrm{in}_v - \mathrm{out}_v - b_v \big)^2 + \rho_{FC} \sum_{e \in E} \big(f_{uv} + f_{vu} + q_e - m\, x_e \big)^2, \end{aligned} \tag{3}$$

with $b_r = -m$, $b_v = 1$ for $v \in R \setminus \{r\}$, $b_v = 0$ otherwise, and $x_e = 1$ for $e \in H$. The sparsity penalty is $\lambda = 0.08$ and the stability penalty $\eta = 0.15$. Because $x_e^2 = x_e$, the stability term is linear in x_e.

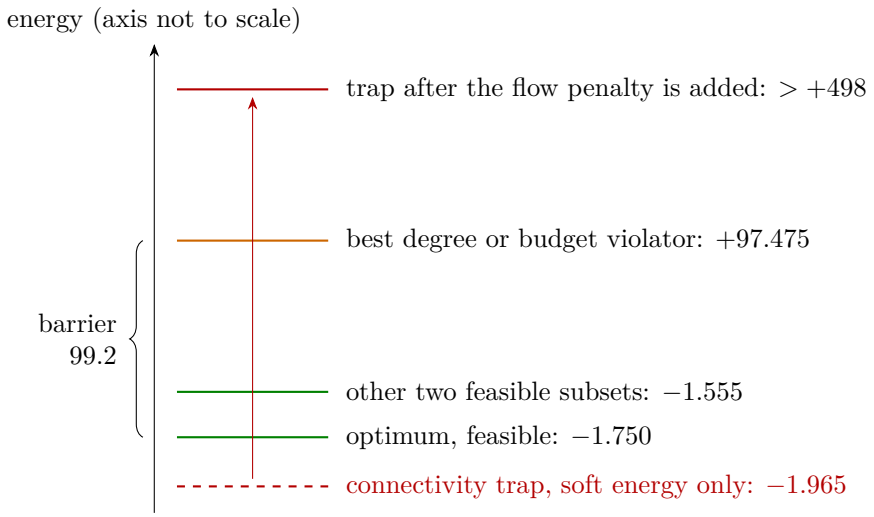


Figure 4: Energy levels of the five-rover example. Without the connectivity terms the trap looks better than the optimum. With the flow penalty it lies far above every feasible subset.

Proposition 1. *The two flow terms are zero for some choice of flows and slacks if and only if every required rover is connected to r in the subgraph of kept edges.*

Proof sketch. If every required rover is connected to r, take a spanning tree of the root's component and send one unit from r to each required rover along its tree path. No arc carries more than m units, conservation holds with the given supplies, and the slack $q_e \in [0, m]$ closes each capacity equality. Conversely, zero penalties mean that m units leave r, one unit is absorbed at every other required rover, and flow only uses kept edges, so each required rover is reachable from r. □

Penalty dominance. We set $\rho_d = \rho_C = 100$ and $\rho_F = \rho_{FC} = 500$. The soft objective of one edge is at most about 1.2 in size, while one unit of violation costs at least 100 and frees at most one edge. Flow violations are penalized quadratically against a linear gain. No violation can therefore pay off, at any problem size, and on 38 random instances the exact bit-level optimum of Eq. (3) never contained a violated constraint.

A worked example. Consider five rovers, the root 0 and rovers 1 to 4, with seven links in range. Link 0–1 is hard-critical because it is rover 1's only tether, the required set is $\{0, 2, 3, 4\}$, the degree caps are $(3, 2, 3, 2, 2)$ and the budget is $K = 12$. Even this small instance needs 61 binary variables (six edge bits, 42 flow bits, 9 degree-slack bits and 4 budget-slack bits), although the decision has only $2^6 = 64$ outcomes. Enumerating all of them gives just three feasible subsets. The optimum keeps $\{0\text{–}1, 0\text{–}2, 2\text{–}3, 2\text{–}4\}$ at energy -1.750, and the other two feasible subsets score -1.555. Thirty of the 61 infeasible subsets break only connectivity, and the best of them, $\{0\text{–}1, 0\text{–}2, 1\text{–}2, 2\text{–}3\}$, has a *lower* soft energy of -1.965 because it keeps the attractive link 1–2, but it cuts rover 4 off from the root. A pruner that looks only at soft scores, or checks connectivity afterwards, would choose it. In the QUBO the flow terms push it above +498. The cheapest subset that breaks a degree or budget constraint sits at +97.475, a barrier of 99.2 above the optimum, which is almost a hundred times wider than the band of feasible energies (Fig. 4). The matrix energy of the optimum, the projected annealer of Section 5 and the exact MILP all give the same answer.

Verification. A second implementation evaluates every term of Eq. (3) directly from the specification, without building Q. On 40 instances with 8 random states each, the matrix energy

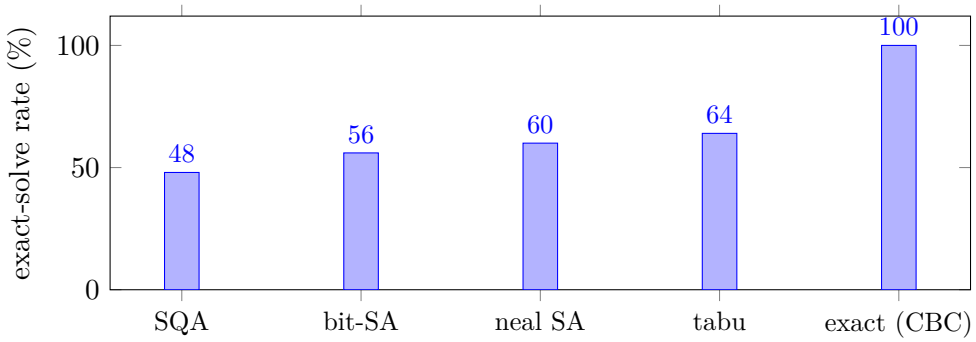


Figure 5: Share of the 25 verifiable instances on which each solver finds the exact optimum of the actual Q matrix: simulated quantum annealing (OpenJij), local simulated annealing, D-Wave's `neal`, tabu search, and the exact linearized solve by CBC that defines the optimum.

$z^\top Qz + \text{const}$ agrees with it to within 2.3×10^{-10}. This is empirical evidence about the implementation and not a formal proof. It does let us attribute a poor answer in the next section to the solver and not to the encoding.

5 Solving the QUBO

5.1 The penalty barrier

A QUBO like Eq. (3) is written to be sampled by an annealer. From a feasible state, a single bit flip either keeps feasibility, which changes the soft objective by at most about 1.2, or breaks a constraint and raises the energy by at least 100. Keeping feasibility is rare, because slack and flow bits have to change together with an edge bit. At a temperature that is useful for the soft landscape, a constraint-breaking move is accepted with probability around $e^{-100/\tau}$, which is effectively zero, so the sampler cannot move between feasible states that differ by more than one coordinated change. In our tests, random restarts of bit-level annealing ended in infeasible states on 19 of 25 instances.

To check that this is not a weakness of one implementation, we gave the actual Q matrices of 25 small instances (four to seven rovers, at most sixteen optional edges, tight caps) to four samplers: our bit-level simulated annealer, D-Wave's `neal`, tabu search and simulated quantum annealing from OpenJij. The ground truth is the exact optimum of the linearized Q matrix solved by CBC, cross-checked against enumeration on instances with at most 60 variables. Each sampler's answer is projected to the edge set it selects before scoring, which helps the samplers. All four miss at least a third of the instances (Fig. 5). Our claim is limited to this penalty-encoded landscape. The exact solver handles the same matrices without any trouble.

5.2 Projected edge annealing

The production solver works on edge sets instead of bit strings. A move either toggles one optional edge or swaps a kept edge for an unkept one, with equal probability. Each candidate is projected back to feasibility: the degree and budget slacks are computed in closed form, a spanning-tree flow is rebuilt from the root to every required rover, and a move that cannot be completed is rejected. In a feasible state all penalties vanish, so the solver still minimizes exactly $H(z)$, only restricted to feasible states, and the Q matrix can still be exported to hardware. Restarts begin from random spanning trees that respect the degree caps, because growing a tree greedily one edge at a time can get stuck when every partial tree is infeasible. The first restart may start from the previous decision restricted to the live graph. After the temperature schedule a deterministic polish alternates single toggles and swaps until no move improves the energy. Since every candidate is feasible, the search never meets the penalty walls.

Table 1: The relaxation ladder of the production pruner, tried from top to bottom. The rung used is logged in every cycle.

Rung	Constraints enforced
full	degree caps, budget, connectivity of reachable required rovers
no-budget	degree caps and connectivity
budget-only	degree caps and budget
unconstrained	degree caps only
keep-all fallback	none; every candidate link is kept

We keep this solver out of the sampler comparison because its moves are richer than bit flips. As an engineering result, it is exact on all 25 verifiable instances (about 2.4×10^4 energy evaluations and 2.1 s per solve), its exact-solve rate on a fixed instance set stays at 87% under every sensitivity perturbation we tried (Section 7.7), and on 20 larger instances with up to 817 binary variables its mean gap to the best of four annealer seeds is 6.9%.

5.3 The relaxation ladder

In a live system the constraints can be jointly infeasible, for example when the budget is below the cheapest connected subgraph. The production pruner therefore tries a sequence of problems and records which one it used in every cycle (Table 1). The required set is the mission roster restricted to rovers reachable from the root, and unreachable mission rovers are counted instead of being dropped silently. One defect is worth reporting: the warm start counts as one restart, so with a single restart and an infeasible warm start no random tree is ever tried, and a solvable instance falls through to the keep-all fallback. We found this while calibrating the 3D engine of Section 8, which now uses at least two restarts.

5.4 The exact MILP baseline

The obvious question for any QUBO pipeline is why the constrained problem is not solved directly. The soft objective is linear, the stability term is linear for binary variables, and the single-commodity flow relaxation is integral for integral data [16], so the problem is a mixed-integer linear program. We solve it with CBC and accept an answer only when optimality is proven. On 20 instances with up to 40 optional edges, the projected annealer reaches the MILP optimum in 95% of cases, with mean normalized gap 7×10^{-5} and worst gap 1.4×10^{-3}, where the gap is $(H_{\mathrm{SA}} - H^*)/\max(1, |H^*|)$. CBC, however, takes about 11 to 15 ms per instance against about 2.1 s for the annealer. On classical hardware the MILP is exact and roughly 100 times faster. The QUBO is still the input format of quantum and quantum-inspired annealers, and the projected annealer is a good classical stand-in for that encoding until such hardware is tested, but it is not the solver we would recommend on a classical machine.

6 Trust-Aware GNN Inference and Assignment

On the kept graph each rover is scored by two layers of trust-weighted message passing. Starting from $h_i^{(0)} = [T_i, B_i, 1 - \ell_i, q_i]$,

$$m_i = \frac{\sum_j a_{ij} h_j}{\sum_j a_{ij}}, \qquad h_i' = \mathrm{ReLU}\big(W_s h_i + W_n m_i + b\big), \qquad s_i = \sigma\big(w^\top h_i^{(2)} + \beta\big),$$

with $a_{ij} = T_{ij} L_{ij}$, $W_s = 0.8\, I$, $W_n = 0.2\, I$, $b = 0$ and readout weights $w = (0.60, 0.15, 0.10, 0.15)$. Pruned edges carry no messages, which is how the pruning decision enters the scores. With equal self and neighbour weights, neighbour information washed out the differences between rovers in a cluster, so the self weight dominates.

The raw score is then adjusted three times. A rover is gated to zero if its trust is below the task minimum (0.25), its battery below 15% or its free compute below the task need (0.3), and if the trust gate would leave a task with no candidate the gate is relaxed for that task. Mission-critical tasks multiply the score by $(0.5 + 0.5\,T_i)$. Finally every score is multiplied by $(0.5 + 0.5\,p_i)$ with p_i from Eq. (2), which prefers rovers with strong delivery paths without letting the root take every task. Tasks are then assigned greedily from the highest score down, with one task per rover per cycle, and each assignment raises the rover's load, which decays over later cycles. With probability $\varepsilon = 0.15$ a task goes to a random eligible rover instead, and these exploration samples are tagged.

Trust follows an exponentially weighted moving average, $T_i \leftarrow \alpha T_i + (1-\alpha)\,\mathrm{obs}_i$ with $\alpha = 0.5$, for rovers that were assigned, while the others drift back towards the prior, $T_i \leftarrow T_i + \delta(0.5 - T_i)$ with $\delta = 0.1$. Without aging, one unlucky failure would leave a rover at 0.25 on the trust gate for good, and without the load feedback the most trusted rover received every task until it ran out of compute; we saw both before adding the fixes. Hidden reliability in the simulator is spatially correlated and drifts over time, so trust has to track a moving target. Pruned links are still tracked: a small state machine marks links as active, suspect, in probation or removed, and links in probation get a freshness floor of 0.4 so they can be re-evaluated when conditions improve. The readout (w, β) can be trained in the cloud by logistic regression on the exploration-tagged samples, which are free of selection bias (Section 7.6). The whole stage costs about 0.2 ms at ten rovers and about 1 ms at fifty.

7 Experimental Evaluation

Setup. All experiments use the open harness in the repository, built around `simulation.py` and covered by a suite of roughly 490 automated checks in `battle_test.py`. The simulator is a synthetic dynamic-graph world with random-walk mobility, distance-based link quality with noise, 5% packet loss that makes link knowledge stale, hidden reliability and scripted faults. It is not a wireless channel model. Unless stated otherwise, scenarios have 10 rovers and 20 cycles. Differences between arms are paired per seed, confidence intervals are 95% paired bootstrap intervals with 10 000 resamples [25], and p-values come from sign-flip permutation tests with 20 000 permutations. Timings were measured on an AMD Ryzen 7 5800H (16 threads, 8 GB RAM) with Python 3.14 and NumPy on its reference linear algebra library.

7.1 Pruner comparison

We compare the QUBO pruner with five baselines on identical scenarios with 16 seeds and drifting reliability (Table 2). Two of the baselines are constraint-aware: *threshold+repair* and *top-K+repair* prune greedily and then run the same repair pass that the simulator offers the QUBO, with the same degree caps, the same budget and exact connectivity of the mission set. Without them, the comparison would only be against pruners that never try to meet the constraints. Besides success we report the offload rate (the share of task slots assigned on the kept graph), the per-cycle delivery-failure rate, and precision and recall of delivery-critical links. An edge is delivery-critical in a cycle if removing it from the full candidate graph strictly lowers the widest-path bottleneck of at least one mission rover, so the labels are recomputed on the live graph in every cycle.

Audit note: the original budget never bound. Our earlier pruner tables used a fixed budget of $K = 15$ units. The relaxation log shows that at this budget the production pruner used the *no-budget* rung in 319 of 320 cycles, because 15 units is less than the cost of any connected subgraph at ten rovers. The QUBO row of those tables was therefore in effect unconstrained. This is the kind of silent failure the log was meant to catch, and we now treat logging the rung

Table 2: Pruner comparison at a binding, feasible budget (0.75× the full-graph cost), 10 rovers, 20 cycles, 16 seeds. Kept is the mean share of candidate links retained, and DC prec. and DC recall are the precision and recall of delivery-critical links. Spearman and AUC compare learned trust with hidden reliability. Violations are mean counts per run for degree / budget / connectivity. Repair times are included in the solve column.

	success	offload	deliv. fail	kept	DC prec.	DC recall
full graph	36.0%	1.000	0.385	100%	0.089	1.000
threshold	37.0%	0.955	0.340	84%	0.097	0.931
top-K	36.4%	0.898	0.309	50%	0.109	0.562
threshold+repair	35.3%	0.996	0.383	44%	0.177	0.817
top-K+repair	35.4%	0.997	0.393	42%	0.187	0.824
QUBO (ours)	31.3%	0.998	0.427	53%	0.116	0.673

	solve (ms)	Spearman	AUC	violations
full graph	none	0.271	0.683	158.8 / 20.0 / 0.4
threshold	< 0.01	0.254	0.624	131.7 / 12.8 / 2.3
top-K	< 0.01	0.251	0.683	53.6 / 0.0 / 10.0
threshold+repair	0.3	0.266	0.683	3.1 / 0.3 / 0.8
top-K+repair	0.1	0.253	0.655	1.1 / 0.0 / 0.9
QUBO (ours)	1470	0.295	0.681	0.0 / 0.0 / 0.4

as a requirement rather than asserting feasibility. All results below use the binding budget of Table 2, where the QUBO solves with every constraint in 320 of 320 cycles.

What the guarantees cost. At the binding budget the QUBO pruner is behind threshold+repair by 4.1 points (CI $[-6.4, -1.9]$, $p = 0.004$) and behind top-K+repair by 4.2 points (CI $[-7.0, -1.5]$, $p = 0.017$). Against the constraint-blind threshold pruner the gap is 5.7 points (CI $[-8.7, -2.9]$, $p = 0.002$), so comparing only with constraint-blind pruners would overstate the cost by about 1.6 points. The gap that remains is still significant. In return, the QUBO is the only arm with no violations at all; the repaired baselines still leave 1.1 to 3.1 degree violations and 0.8 to 0.9 connectivity misses per run. Its learned trust has the best Spearman correlation (0.295 against 0.251 to 0.271), but not the best AUC. The delivery metrics show where it loses. Its delivery-failure rate is 4.4 points higher than threshold+repair (CI $[+1.4, +8.0]$) and 11.8 points higher than top-K (CI $[+7.6, +16.0]$), and its recall of delivery-critical links is 0.67, against 0.82 for the repaired baselines and 0.93 for the threshold pruner. Only the constraint-blind top-K, which keeps half of the links, has lower recall (0.56). The edge-local utility ranks a link by its own quality and not by how much delivery depends on it, so the optimum sometimes drops the redundancy that bottleneck paths need.

7.2 Constraint pressure

If the guarantee is ever going to pay for itself in success, it should do so when the constraints bind hard. We tighten one constraint at a time on identical scenarios, with six pruner arms, 16 seeds per cell and 20 cycles per run: the budget fraction $\beta \in \{0.35, 0.5, 0.75, 1.0\}$, the degree cap $D \in \{2, 3, 4\}$ and the roster size $|R| \in \{3, 5, 8\}$ (Table 3 and Fig. 6).

The result does not match what the guarantee story would predict. At the tightest budget ($\beta = 0.35$) the QUBO ties the best repaired baseline. Here even the repaired arms cannot fully meet the budget (2.9 to 3.0 violations per run), and the QUBO's 5.4 budget residuals are ladder relaxations that the log reports. At every looser setting the QUBO is 2 to 4 points behind, and in the degree and roster sweeps the gap only shrinks as the constraint is relaxed. There is no crossover, even though the constraint-blind threshold pruner breaks the budget in 20 of 20 cycles at $\beta = 0.35$ (12.8 of 20 at $\beta = 0.75$) and exceeds the degree caps 165 times per run at $D = 2$

Table 3: Constraint-pressure sweep: success of the QUBO pruner and of the best repaired baseline at each level, and the paired difference with its permutation p-value.

constraint	level	QUBO	best repaired	QUBO − best
budget β	0.35	34.6%	34.7%	−0.1 pts, $p = 1.00$
	0.50	31.6%	35.2%	−3.7 pts, $p = 0.033$
	0.75	31.3%	35.4%	−4.1 pts, $p = 0.004$
	1.00	31.7%	35.3%	−3.7 pts, $p = 0.020$
degree D	2	30.9%	34.3%	−3.3 pts, $p = 0.15$
	3	32.2%	34.3%	−2.1 pts, $p = 0.10$
	4	33.3%	34.8%	−0.9 pts, $p = 0.52$
roster $\lvert R\rvert$	3	32.2%	34.3%	−2.1 pts, $p = 0.10$
	5	32.4%	35.1%	−2.7 pts, $p = 0.058$
	8	34.6%	37.8%	−3.2 pts, $p = 0.020$

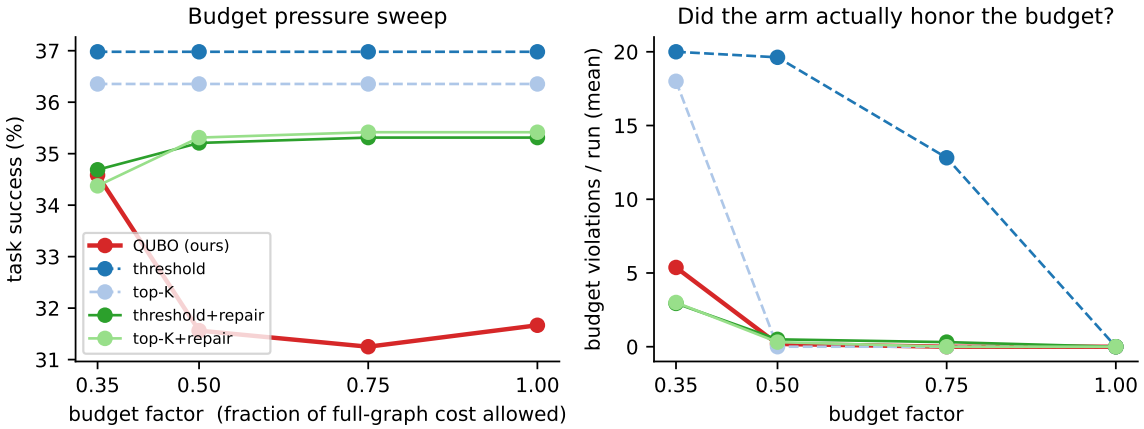


Figure 6: Budget sweep. Left: success of each arm as the budget fraction changes. Right: mean budget violations per run for each arm.

(35 after repair). What does hold at every level is the guarantee: the QUBO has zero degree violations everywhere, budget residuals only where the budget cannot be met, and connectivity misses only in cycles where no pruner could connect the roster (the full graph has the same 0.4 per run). In this simulator the guarantees are worth paying for when breaking a constraint is unacceptable, for regulatory, safety or contractual reasons, and not when success is the only measure.

7.3 Assignment ablation

To see what the GNN adds, we keep the gates, delivery awareness and greedy assignment fixed and replace only the scorer: random choice among eligible rovers, trust-only (the bandit-style baseline), a no-graph readout on the raw features, and the full trust-aware GNN (Table 4). The GNN beats trust-only by 8.1 points, with an interval that excludes zero, and beats random by 13.8 points. Its lead over the no-graph readout is 1.7 points and not significant ($p = 0.74$). At ten-rover density most of the gain comes from the trust model, the gates and delivery awareness, and message passing adds only a small amount that this sample cannot separate from noise.

7.4 Cloud amortization

A solve of more than a second cannot run synchronously on the fog device. The cloud can instead solve every k cycles while the fog device reuses the last decision restricted to the live graph (Table 5). Reusing a decision that is two cycles old costs less than one point of success and halves

Table 4: Assignment ablation with 10 rovers, 20 cycles and 8 seeds. Differences are paired per seed.

policy	success
random	26.0%
trust-only	31.7%
no-graph readout	38.1%
trust-aware GNN (ours)	39.8%
GNN − trust-only	+8.1 pts, CI $[+4.2, +11.7]$, $p = 0.016$
GNN − no-graph	+1.7 pts, CI $[-2.1, +6.9]$, $p = 0.736$

Table 5: Solving in the cloud every k cycles, with 10 rovers, 20 cycles and 6 seeds. Solve time is averaged over all cycles.

solve interval k	success	solves per cycle	amortized solve (ms)
1	30.3%	1.00	1279
2	29.4%	0.50	618
4	26.9%	0.25	273
8	31.9%	0.15	151

the solve load. For $k \geq 4$ the seed-to-seed variation is larger than the effect, so we report the numbers without fitting a curve.

7.5 Scaling and latency

Table 6 reports range-limited sparse graphs from 10 to 50 rovers. Connectivity is enforced only up to 20 rovers because the dense flow encoding grows quadratically, so we do not draw one curve across that point. The projected annealer matches the MILP at ten rovers, is within about 2% of it at twenty, and is ahead of a repaired greedy baseline at every size, by 3.3 energy units at 20 rovers and 12.4 at 50. Fig. 7 shows that the solve takes about five orders of magnitude longer than inference. At this scale the pruner is justified by its guarantees and not by speed, and it belongs in the cloud or on an amortized schedule, as the three-tier design assumes.

7.6 Trust quality, training and faults

Learned trust follows hidden reliability with a Spearman correlation of 0.24 to 0.30 and an AUC of 0.62 to 0.68 during operation. Comparing these numbers across pruners is not clean, because each pruner produces its own stream of observed outcomes, and a common unbiased stream would be needed for a fair ranking. Training the readout on the 78 exploration-tagged samples did not improve held-out success over the fixed weights (32.8% against 36.7%, a difference of −3.9 points, CI $[-11.7, +11.7]$, $p = 0.49$). We report this as a null result; more exploration data would be needed to train the readout properly. In the scripted fault scenario, where the most reliable rover is degraded for four cycles and a packet-loss spike follows, trust in that rover falls, assignment moves away from it, and success returns to its earlier level 3 to 4 cycles after the fault ends.

7.7 Sensitivity

On a fixed instance set the projected annealer is exact on 87% of instances, and it stays at 87% under every perturbation we tested: the trust weight scaled by 1.5, uniform utility weights, penalties divided or multiplied by 5, the sparsity penalty raised to 0.3 and the stability penalty raised to 0.6. This matters because these parameters are currently set by hand.

Table 6: Scaling on range-limited graphs. Connectivity is enforced only for $n \leq 20$ (marked †). E is soft energy, lower is better, and MILP values are exact optima from CBC. At $n = 10$ the repaired greedy edge set is infeasible.

n	$\lvert E \rvert$	vars	solve (s)	E_{proj}	E_{greedy}	E_{MILP}
$10^{\dagger}$	30	427	12.6	-6.39	infeasible	-6.39
$20^{\dagger}$	63	1076	41.4	-15.05	-11.76	-15.36
30	95	194	15.4	-21.12	-16.62	not run
50	169	328	94.5	-36.71	-24.30	not run

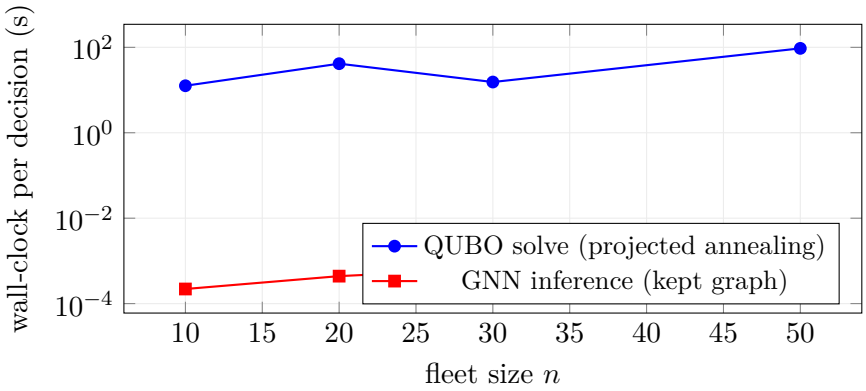


Figure 7: Wall-clock time per decision on the scaling instances. The drop at $n = 30$ is due to connectivity no longer being enforced above 20 rovers.

8 Deployments

Gazebo and ROS 2. The planner also runs as a ROS 2 node against a physics simulation. Four differential-drive rovers ($0.9 \times 0.65 \times 0.25$ m, 3 kg) are spawned in a Gazebo Harmonic world [26], their odometry is bridged into ROS 2 Jazzy [27], and the whole stack runs in one Docker container started with a single command. Once per second the node builds the graph from the live positions (5 m radio range, all four rovers required, root 0, degree cap 3, budget 25 units), prunes it, assigns a critical and a routine task, and publishes its status and assignments as JSON. Link quality depends only on distance, because Gazebo models geometry and not radio propagation, and a short annealing schedule solves the four-rover problem in 12 to 16 ms. On the spawn line-up the pruner keeps all five candidate links in every cycle, which is optimal here because every link is healthy and neither the caps nor the budget bind. Trust visibly steers assignment: rover 3 holds the critical task while it is trusted, fails in cycle 3, drops from 0.92 to 0.50, and the task moves to rover 1 from the next cycle. The node reads odometry but does not yet publish velocity commands, so the rovers stay at their spawn poses and the graph does not change during a run.

An interactive 3D simulation engine. To let people watch the pipeline make decisions, we built a browser-based 3D engine on three.js [28]. The browser handles time, motion and rendering, and a Python planner service wraps the unchanged closed-loop simulator: once per planning cycle the browser sends the live rover positions and the current parameters, the positions replace the simulator's random walk, and the service returns kept links, trust, assignments and outcomes. Rovers move with differential-drive kinematics at a fixed step of 1/120 s. They are updated one at a time, and a translation is accepted only if, for every other rover at its current position, the new distance is at least $2R_b$ or does not shrink; otherwise the rover only turns in place. This rule keeps every pair of footprint circles at least $\min(2R_b, d)$ apart after each step, so rovers never overlap. Every object class is drawn with one instanced call, and the whole scene costs seven draw

calls whatever the fleet size. For live comparisons the service runs paired followers for trust-only, random and no-graph assignment from the same seed and initial state, and for solver comparisons it hands the exact Q matrix of the current cycle, with the same warm start, to bit-level annealing, `neal`, tabu search or simulated quantum annealing. The engine lists only baselines that our pipeline beats in its comparison menu, and it names the MILP and the threshold pruner, which beat it, in a note next to the menu. The motion code has seventeen property tests and the planner service nineteen.

9 Discussion and Limitations

Three findings run against the simple version of our own pitch, and we think they are the most useful results of the study. First, penalty-encoded constrained QUBOs are easy to write and hard to sample: four annealing-family samplers miss a third or more of verifiable optima, so a paper that reports such an encoding without checking solver output against exact answers may mistake a solver failure for a property of the model. Second, on classical hardware the QUBO route is dominated by a direct MILP that is exact and about 100 times faster, so the encoding is justified by hardware targeting and not by classical performance. Third, at small fleet sizes the pruner costs far more time than the inference it protects and gives up 2 to 4 points of success against constrained baselines. Its value is the guarantee, which matters when a violation is not acceptable.

The study has clear limits. Statistical power is modest, with 6 to 16 seeds per arm depending on the experiment, and the margin of message passing over the no-graph readout is not significant. Trust-quality comparisons across pruners are confounded by selection. The simulator is synthetic, with no wireless channel model, no hardware in the loop and no run on quantum hardware; the Q matrices are exported but untested on a quantum processor. The dense Q matrix limits practical fleets to tens of rovers, and the scaling study enforces connectivity only up to 20 rovers. The trained readout is a null result on limited data, and the utility weights and penalties are set by hand, with a sensitivity analysis but no tuning procedure. In the Gazebo world the rovers do not yet move. A four-rover ESP32 testbed has been designed but not built.

The natural next steps follow from these limits: running the exported Q matrices on a quantum annealer and comparing against the certified optima, adding a delivery-aware term to the utility so that links that bottleneck paths depend on are valued, replacing the dense flow encoding with a sparse or hierarchical one, moving the Gazebo rovers with the engine's controller, and building the physical testbed.

10 Conclusion

We built a pipeline that prunes a dynamic rover graph with a constrained QUBO and assigns tasks with a trust-aware GNN, and we audited each part against exact answers and strong baselines. The encoding is verified on finite instances, and once the solver respects feasibility it is exact on every instance small enough to enumerate and within 1.4×10^{-3} of exact MILP optima on larger ones, while the direct MILP remains the better classical solver. Trust-aware assignment improves mission success significantly over trust-only and random baselines in a simulator where success depends on the kept links. The audit also produced negative results that we consider part of the contribution: annealing samplers struggle with penalty-encoded constraints, the MILP dominates the QUBO on classical hardware, and the pruner trades 2 to 4 points of success for its guarantees at every pressure level we tested. It even found a problem in our own earlier comparison, whose budget was silently relaxed in 319 of 320 cycles. The code, test harness, Gazebo world, 3D engine and all reported numbers are available in the repository.

References

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *ICLR*, 2017.

[2] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *NeurIPS*, 2017.

[3] J. Li, S. Tian, R. Wu, L. Zhu, W. Zhao, C. Meng, L. Chen, Z. Zheng, and H. Yin, "Less can be more: Unsupervised graph pruning for large-scale dynamic graphs," *arXiv:2305.10673*, 2023.

[4] C. Liu, X. Ma, Y. Zhan, L. Ding, D. Tao, B. Du, W. Hu, and D. P. Mandic, "Comprehensive graph gradual pruning for sparse training in graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, 35(10):14903–14917, 2024.

[5] L. Wang, W. Huang, M. Zhang, S. Pan, X. Chang, and S. W. Su, "Pruning graph neural networks by evaluating edge properties," *Knowledge-Based Systems*, 256:109847, 2022.

[6] H. Peng, D. Gurevin, S. Huang, T. Geng, W. Jiang, O. Khan, and C. Ding, "Towards sparsification of graph neural networks," in *Proc. IEEE ICCD*, pp. 272–279, 2022.

[7] D. A. Spielman and S.-H. Teng, "Spectral sparsification of graphs," *SIAM J. Comput.*, 40(4):981–1025, 2011.

[8] F. Glover, G. Kochenberger, and Y. Du, "A tutorial on formulating and using QUBO models," *arXiv:1811.11538*, 2019.

[9] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, 2:5, 2014.

[10] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, 220(4598):671–680, 1983.

[11] D-Wave Systems, "Ocean SDK and dwave-neal," software documentation, https://docs.ocean.dwavesys.com/.

[12] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse Ising model," *Physical Review E*, 58(5):5355–5363, 1998.

[13] Jij Inc., "OpenJij: Framework for the Ising model and QUBO," software, https://github.com/OpenJij/OpenJij.

[14] F. Glover, "Tabu search, part I," *ORSA Journal on Computing*, 1(3):190–206, 1989.

[15] J. Forrest et al., "COIN-OR branch and cut (CBC)," software, https://github.com/coin-or/Cbc.

[16] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*, Prentice Hall, 1993.

[17] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *Int. J. Robotics Research*, 32(12):1495–1512, 2013.

[18] T. A. Ahanger, F. Dahan, U. Tariq, and I. Ullah, "Quantum inspired task optimization for IoT edge fog computing environment," *Mathematics*, 11(1):156, 2023.

[19] N. K. Verma and K. J. Naik, "MAG-Q: Multi-agent actor-critic based GRL with quantum inspired forwarding for enhanced task offloading in clustered fog computing environment," *Expert Systems with Applications*, 318:132044, 2026.

[20] B. A. Ali, M. Gregory, and S. Li, "Trust-aware task load balancing in multi-access edge computing based on blockchain and a zero trust security capability framework," *Trans. Emerging Telecommunications Technologies*, 34(12):e4845, 2023.

[21] J. Guo, G. Huang, Q. Li, N. N. Xiong, S. Zhang, and T. Wang, "STMTO: A smart and trust multi-UAV task offloading system," *Information Sciences*, 573:519–540, 2021.

[22] J. Sabater and C. Sierra, "Review on computational trust and reputation models," *Artificial Intelligence Review*, 24(1):33–60, 2005.

[23] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, 47(2–3):235–256, 2002.

[24] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012.

[25] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.

[26] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ IROS*, vol. 3, pp. 2149–2154, 2004.

[27] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, 7(66):eabm6074, 2022.

[28] three.js authors, "three.js: JavaScript 3D library," software, https://threejs.org.